01_CODE_ALONG_LAB

Pintop | 2026 Training

CLASSROOM CODE-ALONG - Validation, Errors & Logging (Backend Design & Engineering)

Pintop Technologies | Class 04 | Instructor and student companion

What we will produce

Recreate a documented excerpt from the **real project** (`app/Http/Controllers/PayTrackController.php`), practise a small safe variation, verify using the complete app or appropriate tool, and record both success and failure evidence. Preserve the original project.

Why: A user submits a form with an empty name and a negative quantity. If your server trusts it, the bad data reaches the database and corrupts reports for months. Today we teach the server to distrust everything and to fail clearly and honestly.

1. Find and read the real file

`codebase/app/Http/Controllers/PayTrackController.php`

The excerpt is in `REFERENCE EXCERPT.php`. Its job is to give you something to type, trace and understand. It is not guaranteed to run on its own: it is part of a larger file. Do not overwrite project source with a fragment.

2. Set up the real project

```
cd "Backend Design & Engineering/codebase"
composer install
cp .env.example .env
php artisan key:generate
php artisan migrate --seed
php artisan serve --port=8000
```

3. Type in stages

Stage 1

Say: "I am writing a small section of the actual project. Look for the inputs and names; we will connect it with the next block."

```
use Illuminate\Validation\ValidationException;
class PayTrackController {
 public function health(){try{DB::select('SELECT 1');return response()->json(['status'=>'ready','dependenci
 public function register(Request $r){
  $v=$r->validate(['name'=>'required|string|min:2|max:120','email'=>'required|email|unique:users,email','pa
  $created=DB::transaction(function()use($v){
   $id=DB::table('users')->insertGetId(['name'=>$v['name'],'email'=>strtolower($v['email']),'password'=>Has
   DB::table('wallets')->insert(['id'=>(string)Str::uuid(),'user_id'=>$id,'currency'=>'NGN','balance_minor'
   return $id;
  });
  return response()->json(['user'=>DB::table('users')->where('id',$created)->first(['id','name','email','ro
```

Stage 2

Say: "I am writing a small section of the actual project. Look for the inputs and names; we will connect it with the next block."

```
        }
        public function createCustomer(Request $r){return $this->register($r);}
        public function customer(Request $r,int $id){
         $uid=$r->attributes->get('training_user_id');
         if ($uid!==$id)return response()->json(['message'=>'Customer not found'],404);
         $user=DB::table('users')->where('id',$id)->first(['id','name','email','role']);
         return $user?response()->json(['data'=>$user]):response()->json(['message'=>'Customer not found'],404);
        }
        public function wallet(Request $r,string $id){
         $row=DB::table('wallets')->where('id',$id)->where('user_id',$r->attributes->get('training_user_id'))->fir
         return $row?response()->json(['data'=>$row]):response()->json(['message'=>'Wallet not found'],404);
```

Stage 3

Say: "I am writing a small section of the actual project. Look for the inputs and names; we will connect it with the next block."

```
        }
        public function ledger(Request $r,string $id){
         $wallet=DB::table('wallets')->where('id',$id)->where('user_id',$r->attributes->get('training_user_id'))->
         if (!$wallet)return response()->json(['message'=>'Wallet not found'],404);
         $limit=min(100,max(1,(int)$r->query('limit',20)));
         return response()->json(['data'=>DB::table('ledger_entries')->where('wallet_id',$id)->orderByDesc('id')->
        }
        public function transfer(Request $r){
         $data=$r->validate(['source_wallet_id'=>'required|uuid','destination_wallet_id'=>'required|uuid|different
         $key=$r->header('Idempotency-Key');
```

4. Complete these activities

1. **Trace the request from route declaration through controller or handler to response.**
2. **Add a small local test request or safe demonstration route in a disposable branch; predict status and JSON.**
3. **Send a success request and a deliberately invalid request and explain both HTTP results.**

5. Check, debug and record

```
       php artisan route:list --path=api
       php artisan test
```

Expected check: If the local app starts or its tests pass, capture evidence of the specific behaviour tied to this class, not merely an application startup. Test one failed or edge input too. Log the exact error if blocked.

6. Marking

- 4/20: explain concept and input/output in plain language
- 4/20: accurate typed or edited work
- 4/20: local proof and explanation
- 4/20: safe variation and wrong-case recovery
- 4/20: short documented submission

Lab ethics: no unapproved public scanning, no third-party tokens and no real banking customer data.